CSP-203: Software Tools & Technologies Lab



Plotting Tools: GNU Plot
(Lab Session – 14)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

- **Installing GNU plot in linux:**

  1. sudo apt-get update
  2. sudo apt-get upgrade
  3. sudo apt-get install gnuplot

- **Checking the version of GNU plot:**

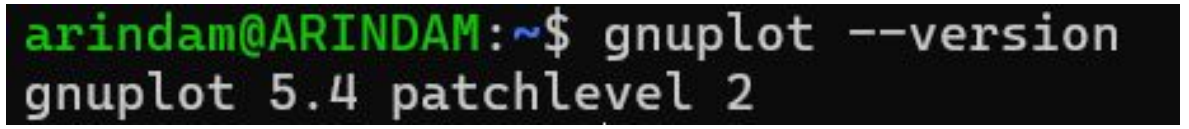
  gnuplot --version

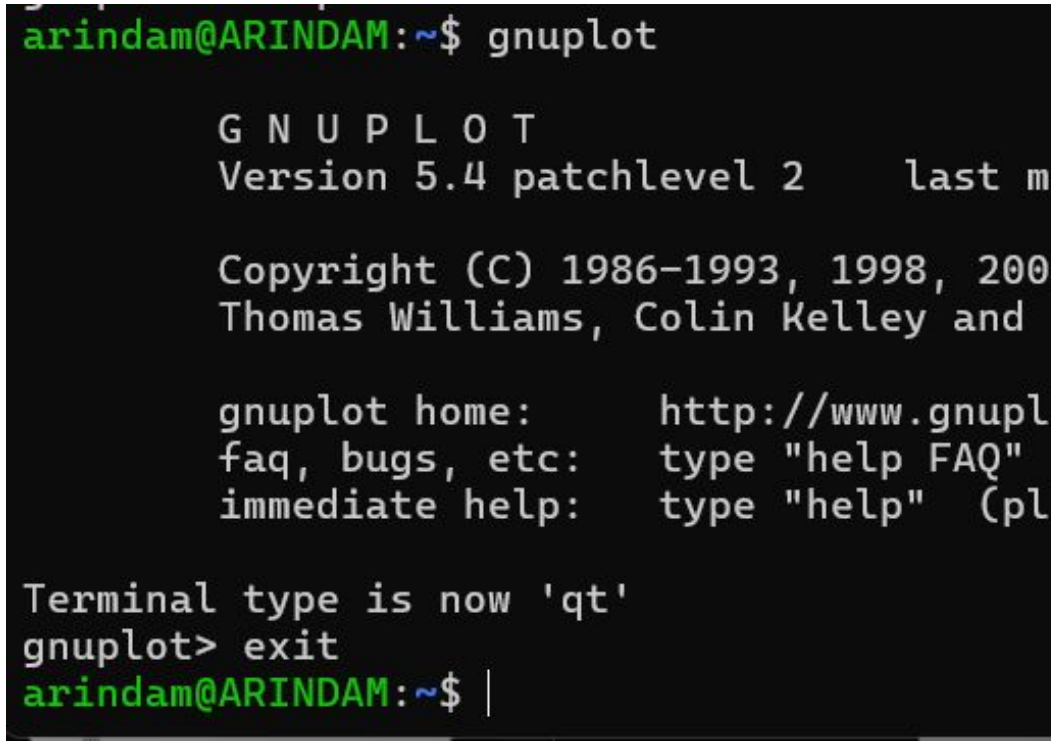
- **How to run GNU plot program:**

  Just type: gnuplot on your terminal.

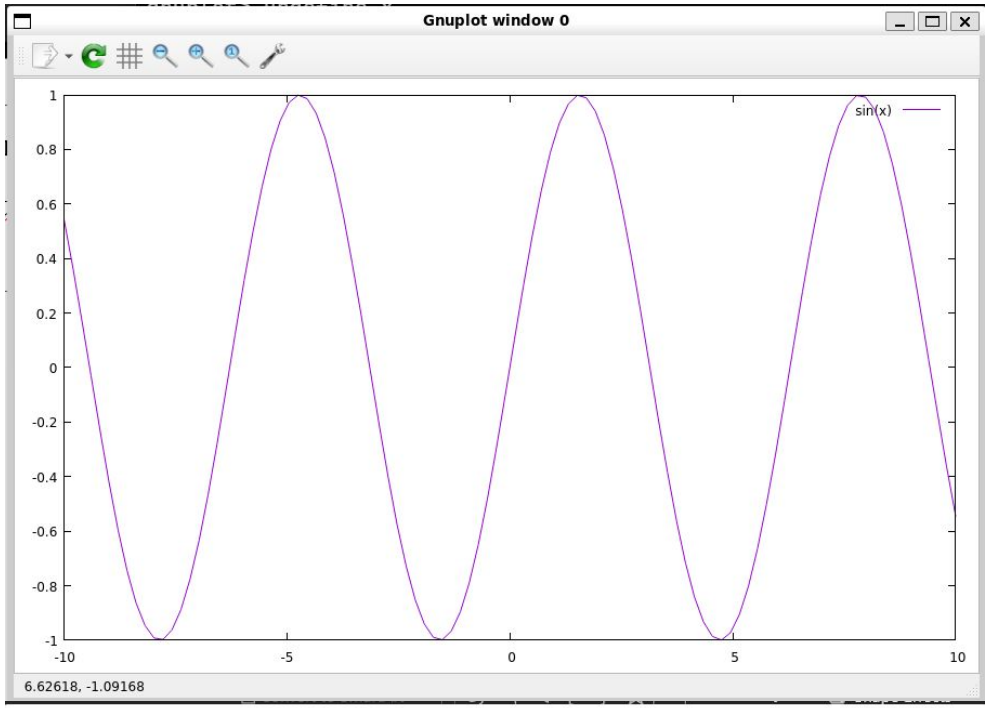
- **How to exit from GNU plot program:**

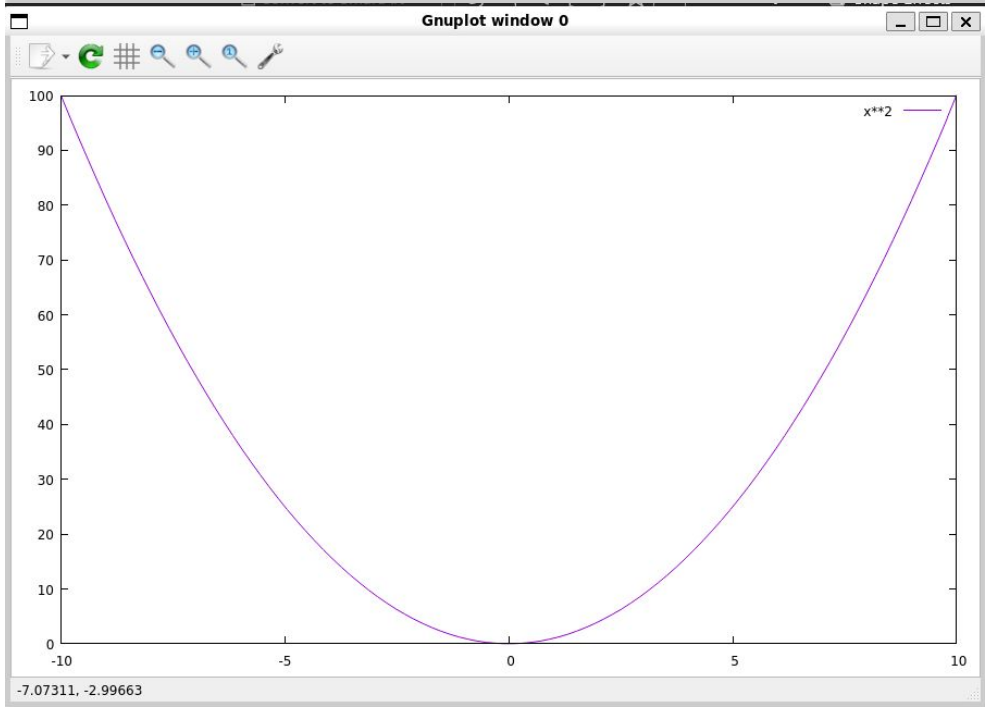
  Type: exit on your terminal





- **Plotting a function:**
  - Plot function_with_variable
  - Example: plot sin(x)
  - Example: plot x**2





- **Variables:**
  - Storing in a variable:
    - a = 5 (Storing integer)
    - a = 4.789 (Storing float)
    - a = {3,4} to store complex(3+4i)
    - a = “STTL”
- **Printing variables:**
  - print “The store value of a is: ”, a

- **Clear Terminal in GNU plot:**
  - Command: !clear
- **Undefine variable:**
  - Command: undefine variable_name

```
gnuplot> a = 25
gnuplot> print a
25
gnuplot> undefine a
gnuplot> print a
         undefined variable: a

gnuplot>
```

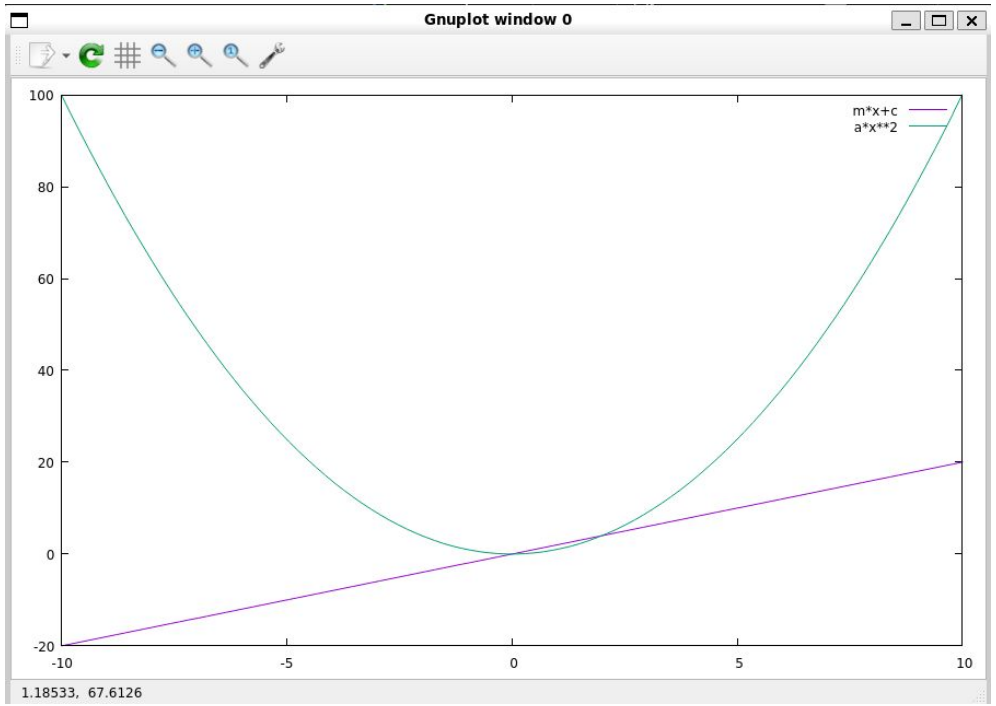
```
gnuplot> x = 2
gnuplot> print (x*x)
4
gnuplot>
```

```
gnuplot> a = {3,5}
gnuplot> print "Real part of a is: ", real(a)
Real part of a is: 3.0
gnuplot>
```

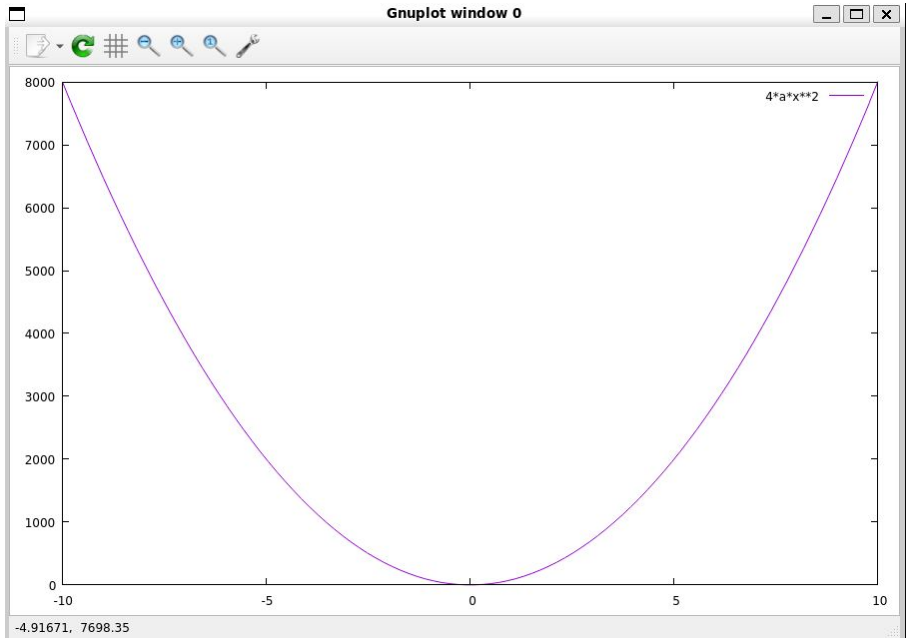
❑ Replot:

- When we call the plot again after plot other function, the previous plot gets erased and the newly called plot is again generated.
- What if we want to plot second function while the first plot is present in the figure?
- The solution is replot.
- We can plot as many functions as we want in a single figure.

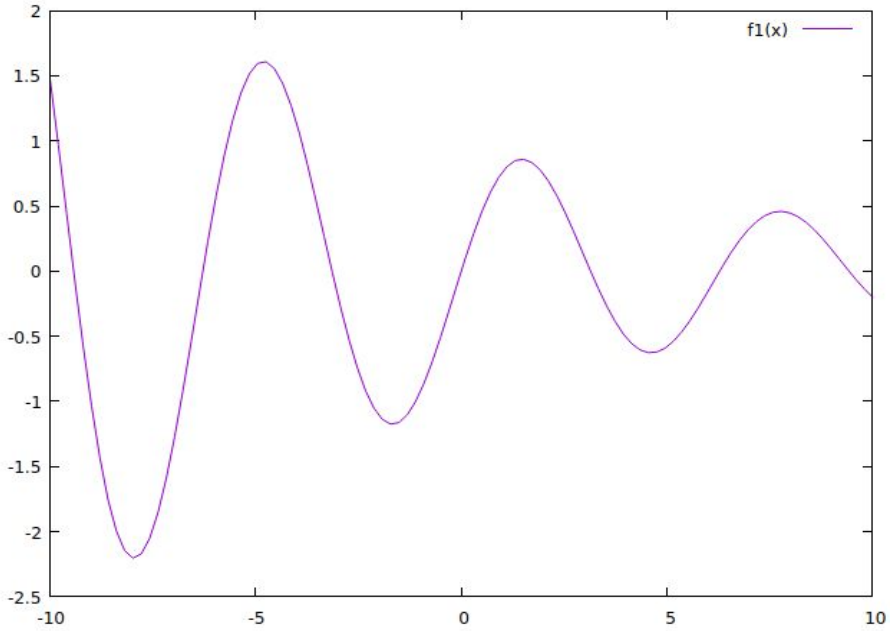
```
gnuplot> m=2
gnuplot> c=0
gnuplot> plot m*x+c
gnuplot> a=1
gnuplot> replot a*x**2
gnuplot>
```



- **Plot $y = 4ax^2$ where a = 8:**
  - a = 2
  - plot 4*a*x**2



- **Plot function** $f_1(x) = sin(x) \cdot e^{-0.1x}$
  - f1(x)=sin(x)*exp(-0.1*x)
  - plot f1(x)



Set terminal

- The terminal defines the output format or device for plots.

Examples:

- **Interactive window** (default: qt or wxt)
- **Image files** (png, jpeg)
- **Document formats** (pdfcairo, eps)

- By default is qt (qt cross-platform interactive terminal)
  - set terminal qt
  - set terminal qt 1 // Opens a separate interactive window with ID = 1.

- *Change terminal*

```
set terminal png          # change terminal
set output "plot.png"     # plot filename
plot sin(x)               # plot function
unset output              #  file close
```

- **Saving figure in GNU-plot:**
  - To save a figure in gnu-plot, you need to specify the terminal type (Output format) before your plot.
  - General Procedure:
    - set terminal pngcairo size 800,600 enhanced font 'Arial,10’

      set output 'output.png’

      plot sin(x)

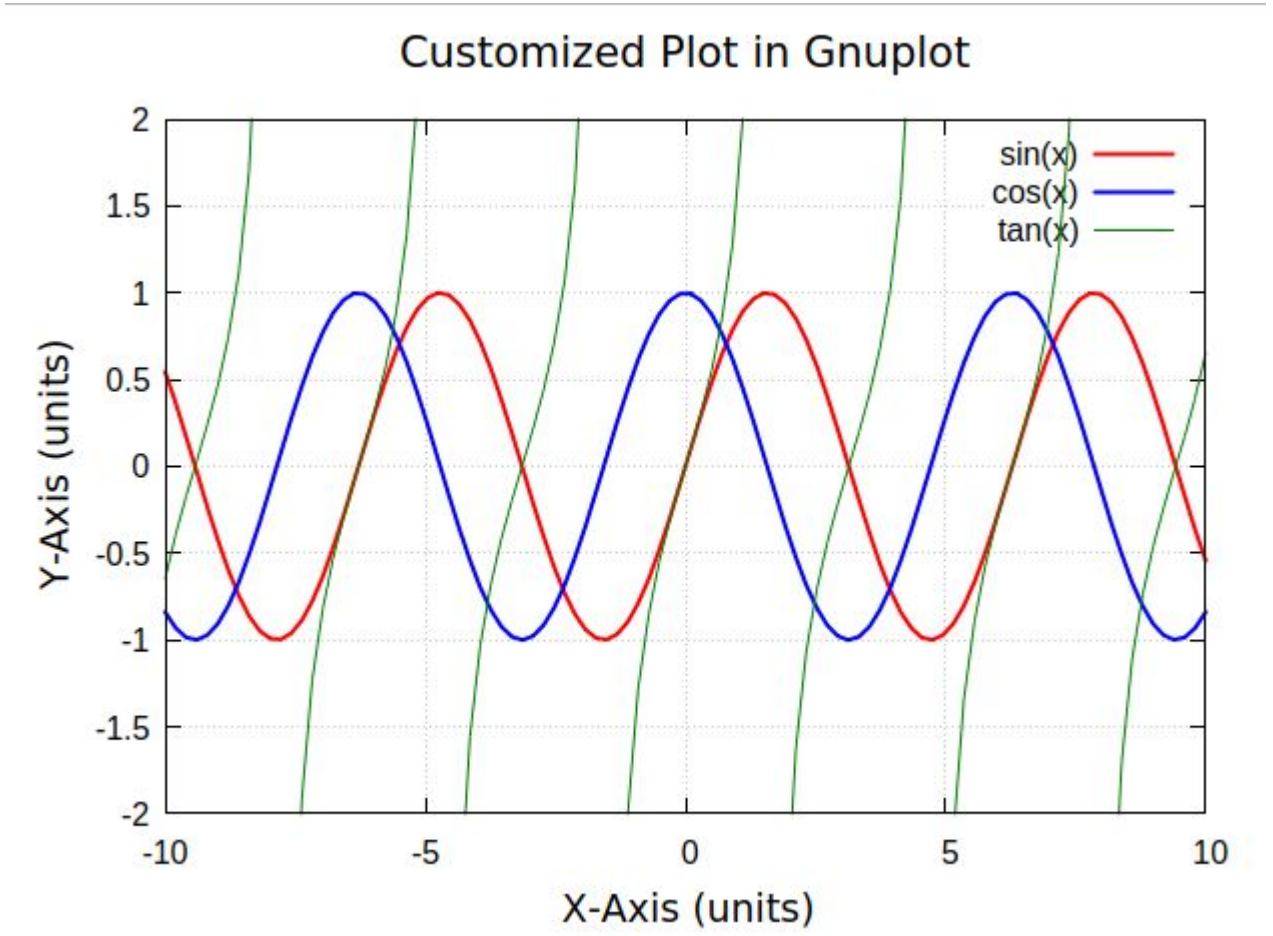
      unset output
  - Things to remember:
    - Always use output command before the plot command.
    - Always unset output after plotting to flush the file and avoid corruption.

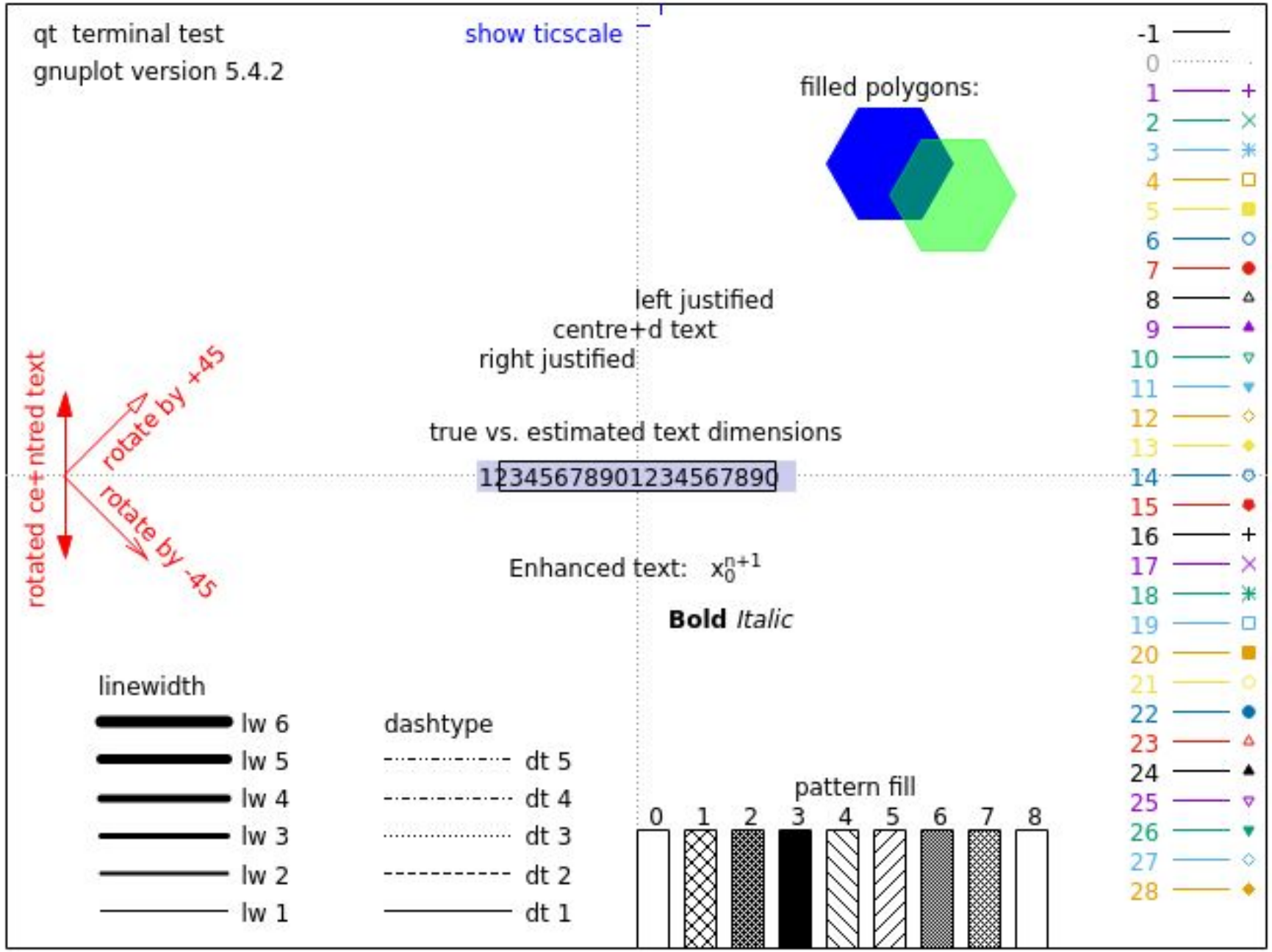
```
# Demo Gnuplot Script: Customizing Plot Appearance
set terminal qt font "Arial,12"       size 800,600            # Set output terminal
# Axis ranges
set xrange [-10:10]       # Fix x range
set yrange [-2:2]         # Fix y range

# Margins
set lmargin 10
set rmargin 5
set tmargin 3
set bmargin 4

# Labels and Title
set xlabel "X-Axis (units)" font "Times-Bold,14"
set ylabel "Y-Axis (units)" font "Times-Italic,14"
set title  "Customized Plot in Gnuplot" font "Helvetica-BoldItalic,16"

# Grid lines for clarity
set grid
set key top right    //legend
# Plot with customized styles
plot sin(x) lt 1 lc rgb "red"   lw 2 title "sin(x)", \
     cos(x) lt 2 lc rgb "blue"  lw 2 title "cos(x)", \
     tan(x) lt 3 lc rgb "#008000" lw 1 title "tan(x)"
pause -1 "Press Enter to close"
```




qt terminal test
gnuplot version 5.4.2
show ticscale
filled polygons:
left justified
centre+d text
right justified
rotated ce+ntred text
rotate by +45
rotate by -45
true vs. estimated text dimensions
12345678901234567890
Enhanced text: x_0^{n+1}
Bold *Italic*
linewidth
lw 6
lw 5
lw 4
lw 3
lw 2
lw 1
dashtype
dt 5
dt 4
dt 3
dt 2
dt 1
pattern fill
0 1 2 3 4 5 6 7 8
-1 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28

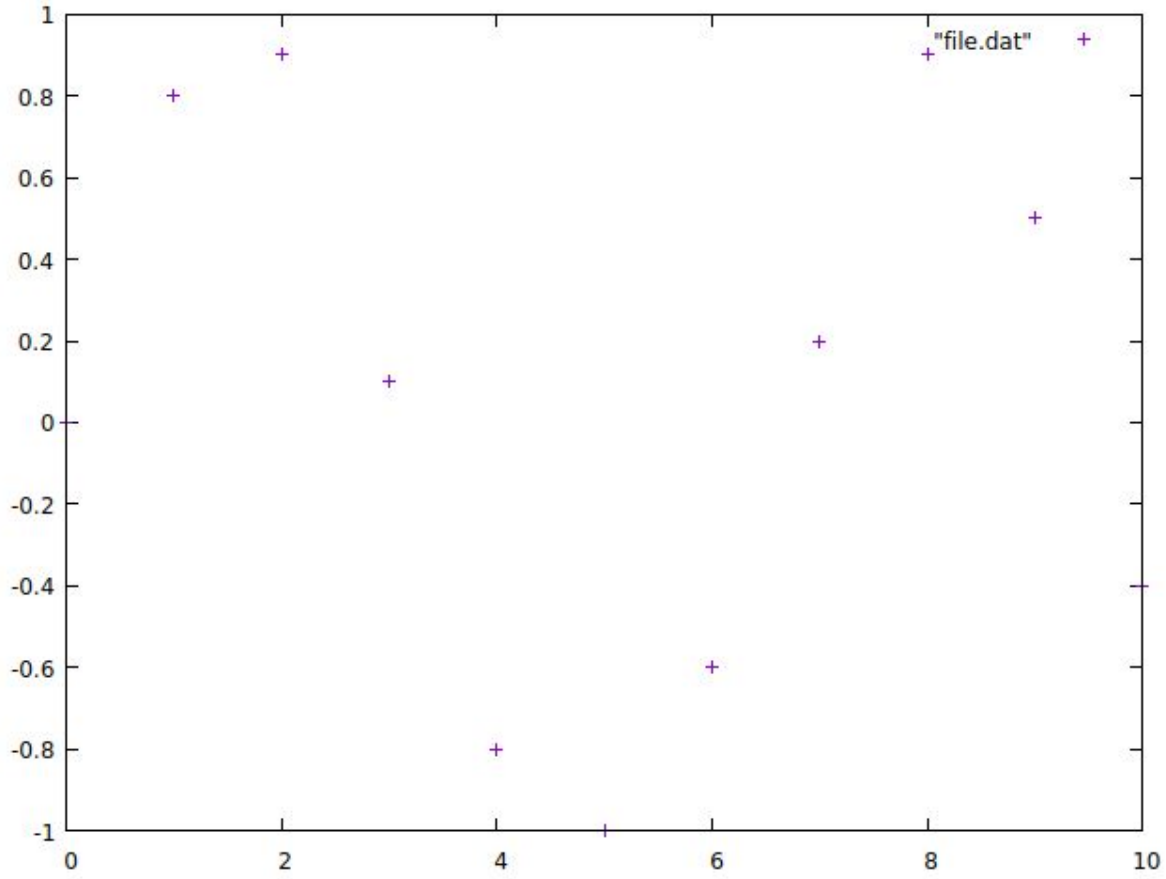
- **ASCII-Datafiles in multi-column format:**

file.dat

```
# Comments can be placed here
x0  y0  z0   ...
x1  y1  z1   ...
.    .   .   ...
xN  yN  zN   …
```

- To plot a multi-column datafile using the 1st column for the abscissa and the 2nd column as the ordinate, use **:** plot “file.dat” using 1:2
- Add a second plot using 1st (=x) and 3rd (=z) columns: replot “file.dat” using 1:3
- If the “using” keyword is not specified, 1st and 2nd columns are assumed: plot “file.dat”

plot “file.dat” using 1:2



# X	Y1	Y2
0	0.0	1.0
1	0.8	0.6
2	0.9	0.1
3	0.1	-0.5
4	-0.8	-0.9
5	-1.0	-0.2
6	-0.6	0.7
7	0.2	0.9
8	0.9	0.3
9	0.5	-0.7
10	-0.4	-1.0

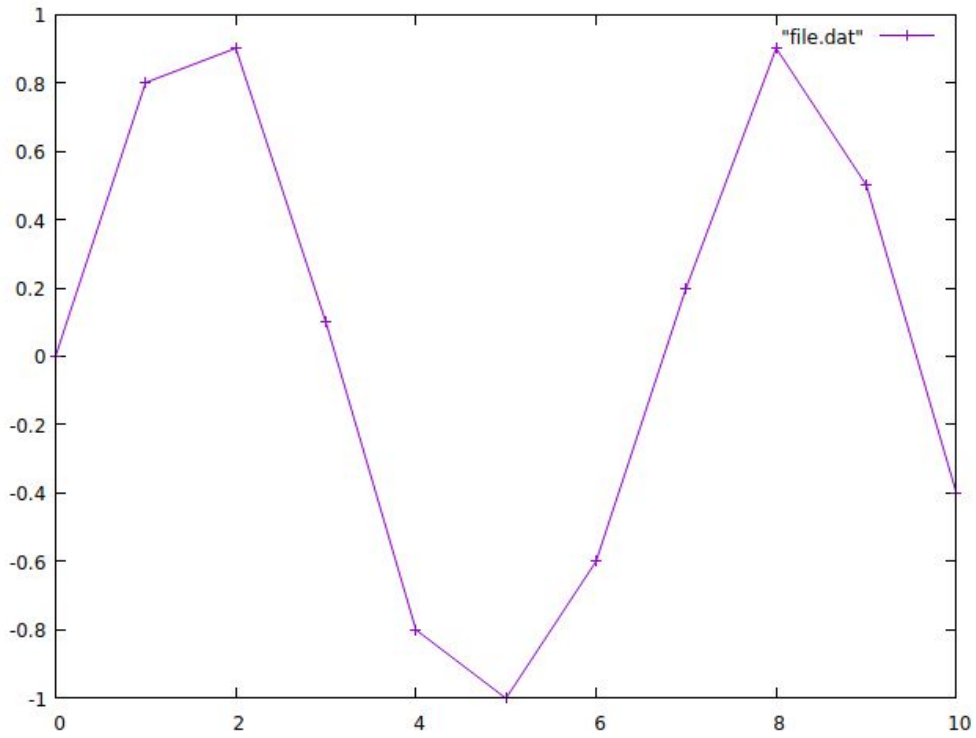
file.dat

- Data file separator: By default, gnuplot treats tabs and white spaces as field value separator.
- If we want to specify the field value separator manually, we have to use this command: **set datafile separator “<separator_name>"**

- **When plotting datafiles, gnuplot uses symbols and join symbol with lines:**

  plot “file.dat” using 1:2 with lines

- plot “file.dat” using 1:2 with linespoints




```
# X   Y1    Y2
0     0.0   1.0
1     0.8   0.6
2     0.9   0.1
3     0.1  -0.5
4    -0.8  -0.9
5    -1.0  -0.2
6    -0.6   0.7
7     0.2   0.9
8     0.9   0.3
9     0.5  -0.7
10   -0.4  -1.0
```

file.dat

Country	2000	2010	2020
India	1050	1230	1380
China	1260	1340	1400
USA	282	309	331
Brazil	174	196	213
Nigeria	122	159	206

file.dat

- # Single point plot

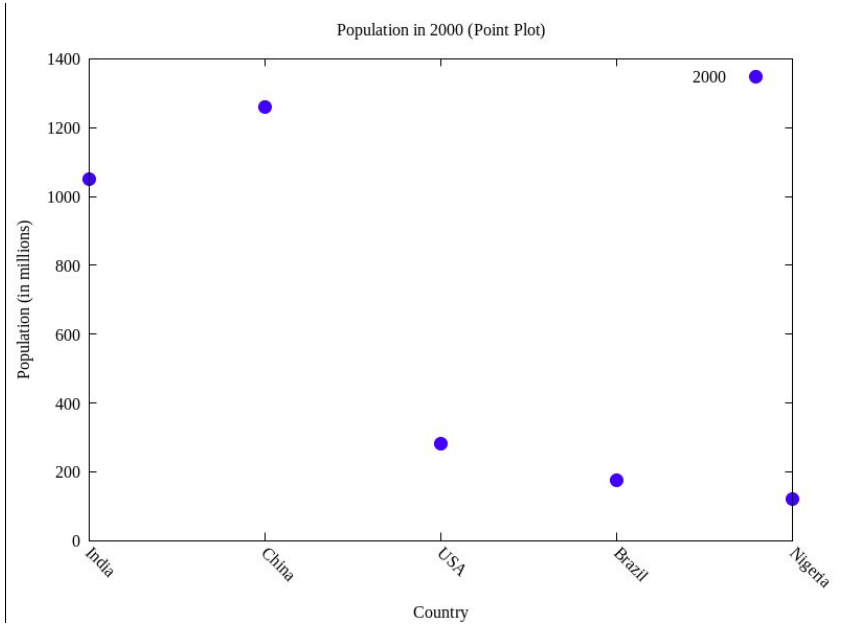
```
set terminal pngcairo size 800,600 enhanced font "Times New Roman,12"
set output "population_points.png"

set title "Population in 2000 (Point Plot)"
set xlabel "Country"
set ylabel "Population (in millions)"

set xtics rotate by -45

plot "file.dat" every ::1 using 2:xtic(1) with points pt 7 lc rgb "blue" title "2000"

pause -1
```



Creating script to execute gnu-plot:

- Create a script file: filename.gp
- Write the commands in the script file:

```
# Set the terminal type and output file
set terminal pngcairo size 800,600
set output 'line_graph.png’
# Set plot title and axis labels
set title 'Line Graph Example’
set xlabel 'X-Axis’
set ylabel 'Y-Axis’
# Plot the data
plot 'data.dat' using 1:2 with lines title 'Data Line’
```

- Execute the script: gnuplot line_plot.gp

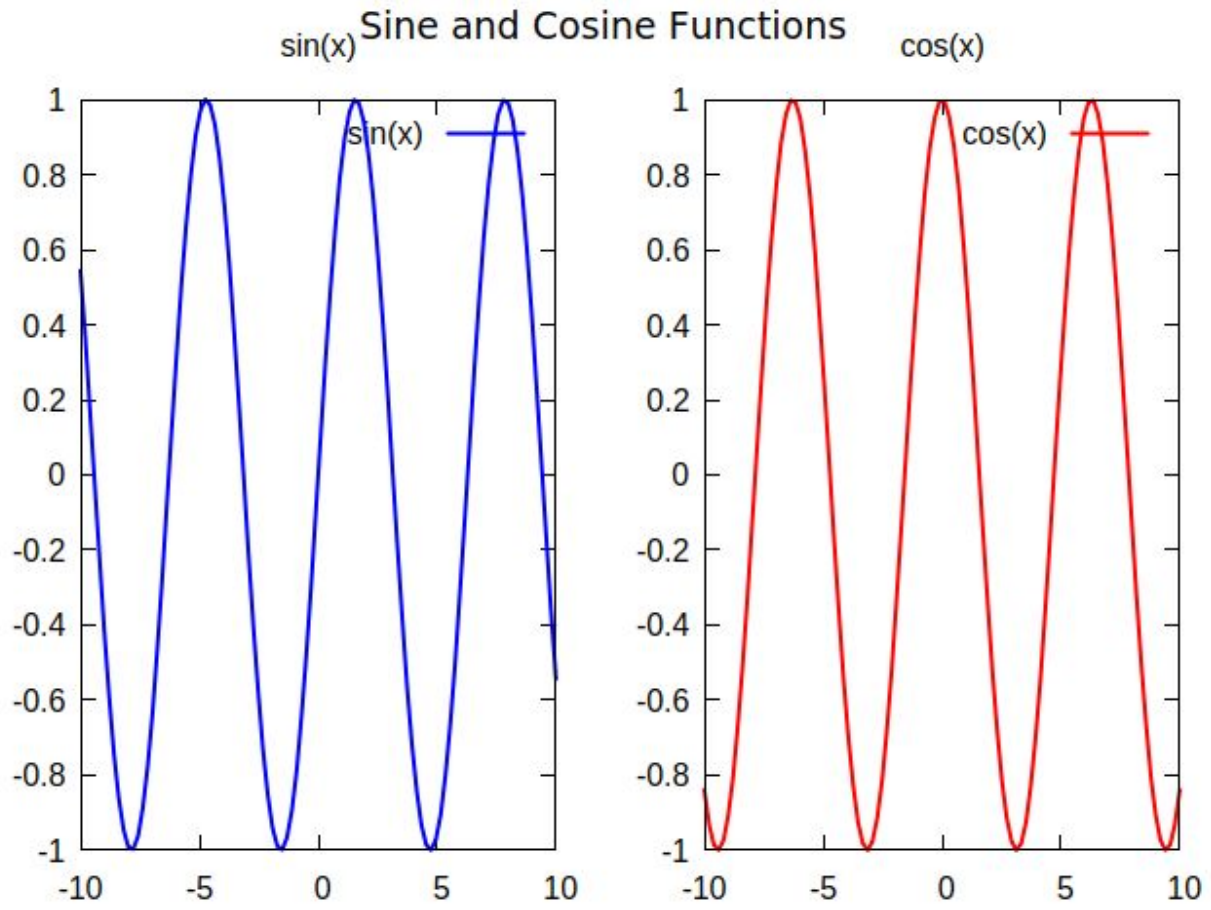
First Example – Two Plots (Manual Multiplot)

```
set term qt font "Arial,12"
set multiplot title "Sine and Cosine Functions" font "Helvetica-Bold,14"

# First plot
set origin 0.0,0.0          # (x,y) position in [0,1] scale
set size 0.5,1.0            # width=50%, height=100%
set title "sin(x)"
plot sin(x) with lines lw 2 lc rgb "blue"

# Second plot
set origin 0.5,0.0          # shift to right half
set size 0.5,1.0
set title "cos(x)"
plot cos(x) with lines lw 2 lc rgb "red"

unset multiplot
pause -1 "Press Enter to close"
```



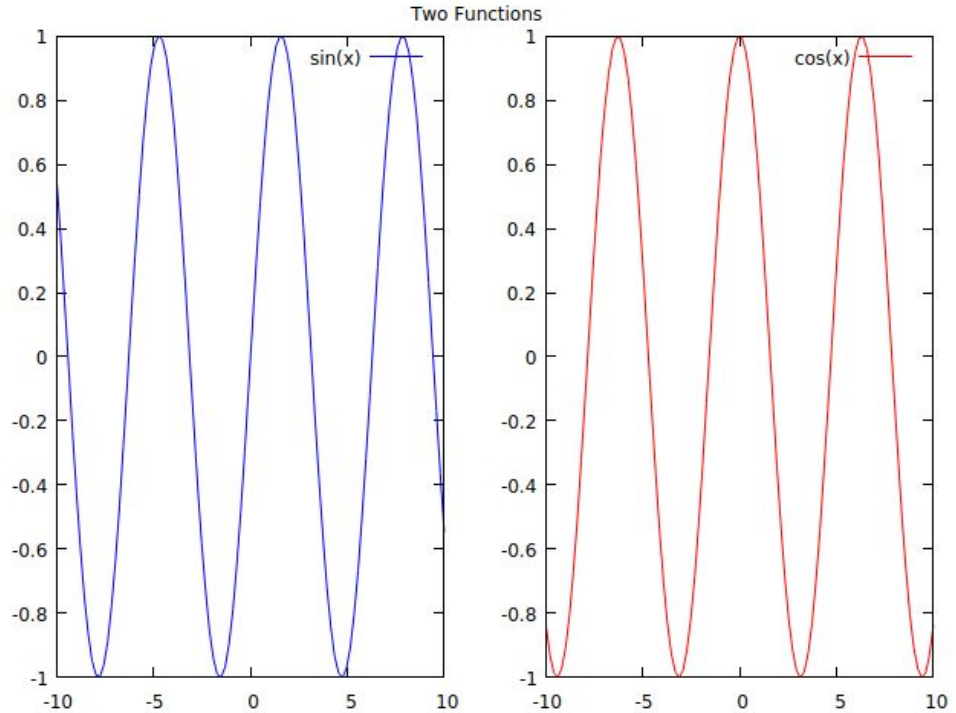
- **Multiplot**
  - Normally, Gnuplot shows one graph per window.
  - With set multiplot, you can display multiple graphs in the same window/output.
  - Layouts:

    set multiplot → you control positions manually.

    set multiplot layout <rows>,<cols> → automatic grid layout.

❑ Using Layout (Simpler Way)

```
set multiplot layout 1,2 title "Two Functions"
plot sin(x) title "sin(x)" with lines lc rgb "blue"
plot cos(x) title "cos(x)" with lines lc rgb "red"
unset multiplot
pause -1 "Press Enter to close"
```



❑ 2×2 Grid

```
set multiplot layout 2,2 title "2x2 Multiplot Example"

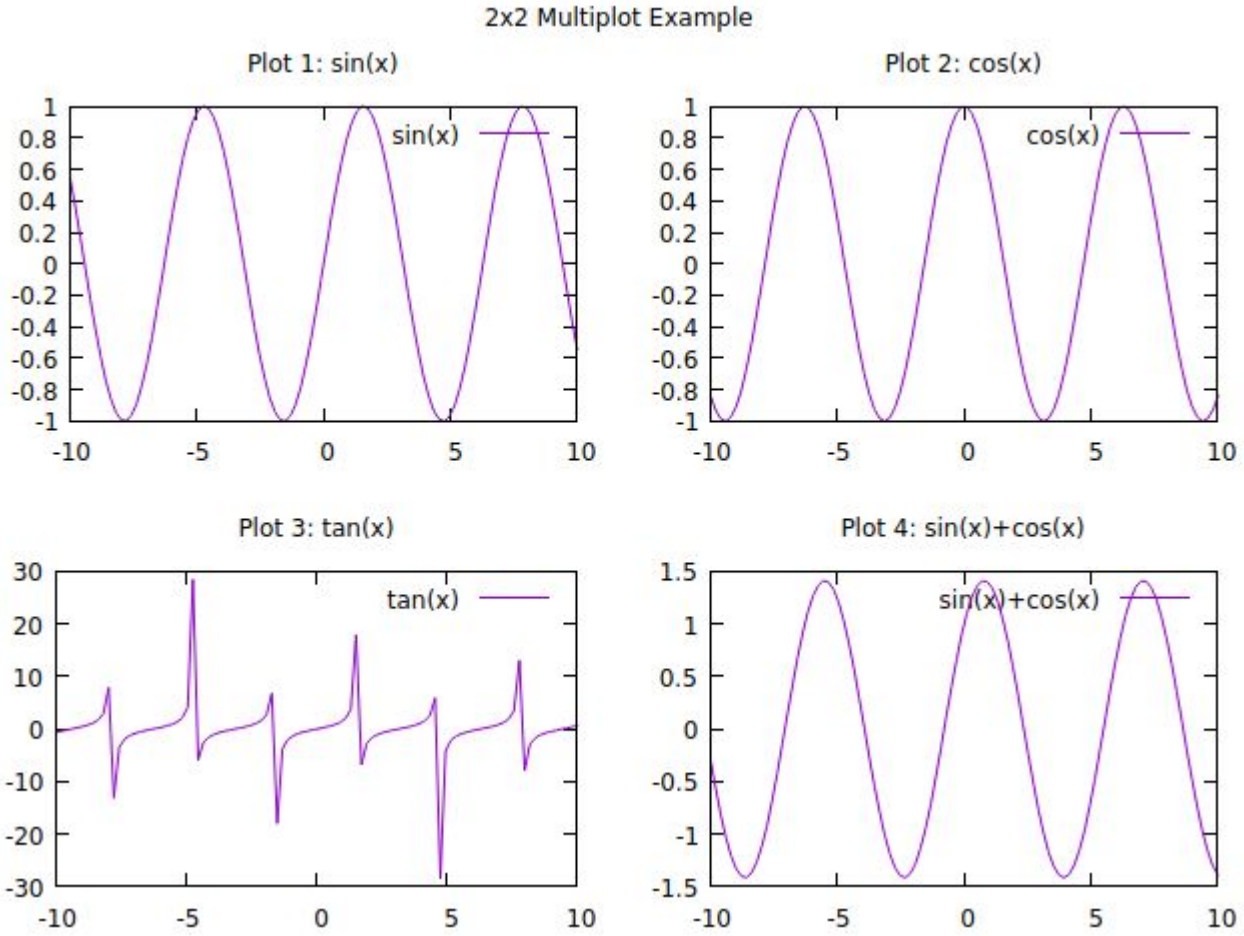
set title "Plot 1: sin(x)"
plot sin(x)

set title "Plot 2: cos(x)"
plot cos(x)

set title "Plot 3: tan(x)"
plot tan(x)

set title "Plot 4: sin(x)+cos(x)"
plot sin(x)+cos(x)

unset multiplot
pause -1 "Press Enter to close"
```




THANK
YOU